



多模态大模型项目期末报告

题目： 基于多模态检索增强生成的文档理解
问答系统

小组成员 刘天翔 杨毅涵 郭翊涵

学 号 23320104 23330154 22308043

院 系 计算机学院

专 业 计算机科学与技术

2026 年 6 月 17 日

摘 要

面向 PDF、扫描文档和图表型页面的问答任务，传统纯文本 RAG 方法依赖 OCR 或文本抽取结果，容易丢失版面结构、图表数值和视觉区域位置，导致复杂文档中的证据定位与回答生成不稳定。开题报告中，本项目计划实现一个集文档解析、视觉定位、自然语言理解、模型路由和可解释溯源于一体的“文档智能助手”。围绕这一目标，本文完成了一个可运行、可评测、可交互的多模态检索增强生成系统，支持 PDF/图片上传、页面渲染、text/table/figure/code 等结构化 chunk 构建、文本-图像统一 embedding、向量检索、重排、自动路由、多模态回答生成以及前端 PDF 证据高亮预览。

在工程实现上，系统采用 FastAPI 与 React 构建交互式文档问答界面，后端以 PyMuPDF 完成页面解析、区域裁剪和表格 Markdown 生成，以 BGE-M3 构建文本向量表示，以轻量图像特征或可选 Native ViT 形成图像向量，并通过统一文本-图像 embedding 接口实现多源 evidence 融合。检索阶段使用向量索引与 BGE-reranker-v2-m3 进行证据重排，生成阶段通过 Qwen3-VL-8B-Instruct 对图表、表格、广告和页面局部区域进行视觉理解。对于开题报告中提出的强化学习模型路由，最终项目实现了 GRPO-style auto router、候选 VLM 打分和策略更新钩子，但未进行真实强化学习训练；对于多模态向量化与溯源展示，则完成了结构化 chunk、页面图、局部 crop、bbox 坐标和 PDF 高亮联动的工程闭环。

实验部分在 DocVQA 与 ChartQA 子集上比较 Text-RAG 与 MM-RAG。结果表明，在 20 个 DocVQA 与 20 个 ChartQA 样本上，MM-RAG 总体严格 EM 达到 0.9750，ANLS 达到 0.9750，显著高于 Text-RAG 的总体 EM 0.3750 与 ANLS 0.4265。其中 ChartQA 子集 MM-RAG 达到 20/20，说明局部高清视觉 crop 与图表专用路由能有效弥补纯文本检索对视觉数值和图表结构的缺陷。最后，本文分析了系统在复杂表格行列对齐、OCR 噪声、模型调用成本与长文档扩展方面的局限，并从 AGI 视角讨论多模态文档智能助手在真实办公、科研和知识管理中的潜力与瓶颈。

关键词：多模态大模型, 检索增强生成, 文档理解, DocVQA, ChartQA, Qwen3-VL

ABSTRACT

This report presents a multimodal retrieval-augmented document question answering system for PDFs, scanned documents, and chart-rich pages. Following the proposal, the project aims to build a document assistant that integrates document parsing, visual localization, natural-language understanding, route selection, and explainable citation. The final system combines page rendering, structured text/table/figure/code chunks, local visual-region cropping, unified text-image embeddings, vector retrieval, reranking, auto routing, and Qwen3-VL based answer generation. The implementation provides an end-to-end web demo with FastAPI and React, supporting file upload, parsing progress, knowledge-base chunks, continuous PDF preview, and evidence highlighting.

The backend integrates PyMuPDF for document parsing, BGE-M3 for text embeddings, lightweight image features or optional Native ViT for image embeddings, BGE-reranker-v2-m3 for reranking, and Qwen3-VL-8B-Instruct for multimodal reasoning over local document regions. The GRPO-based learned router proposed at the beginning is implemented as a GRPO-style auto router with candidate scoring and a policy-update hook, but it is not fully trained due to limited data and time. To improve transparency, every answer is grounded with structured citations, including page numbers, chunk identifiers, and bounding boxes. The frontend further links selected chunks with highlighted regions in the PDF preview.

Experiments are conducted on subsets of DocVQA and ChartQA. On 20 DocVQA and 20 ChartQA samples, the multimodal RAG pipeline achieves an overall strict EM of 0.9750 and ANLS of 0.9750, while the text-only RAG baseline obtains an overall EM of 0.3750 and ANLS of 0.4265. These results demonstrate the effectiveness of local high-resolution visual crops and chart-specific routing for document understanding. The report also discusses remaining limitations, including table row alignment, OCR noise, model cost, and long-document scalability, and reflects on the potential and bottlenecks of multimodal document assistants from an AGI perspective.

Keywords: multimodal large language model, retrieval-augmented generation, document understanding, DocVQA, ChartQA, Qwen3-VL

目 录

1	绪论	1
1.1	任务背景	1
1.2	问题定义	1
1.3	开题目标与完成概览	2
1.4	系统贡献	2
1.5	报告结构	3
2	相关技术与功能对应	4
2.1	组件功能概览	4
2.2	Embedding 与 Fusion	4
2.3	向量检索与 Reranker	5
2.4	GRPO 与自动路由	5
2.5	VLM 多模态生成	6
2.6	PDF 高亮与可解释溯源	6
2.7	小结	7
3	方法与系统实现	8
3.1	总体架构	8
3.2	文档解析实现	8
3.3	Embedding 与 Fusion 实现	9
3.4	向量检索与 Reranker 实现	9
3.5	路由决策实现	10
3.6	VLM 与局部视觉区域实现	10
3.7	FastAPI 服务层实现	11
3.8	前端 PDF 预览与高亮实现	11
3.9	代码结构	12
4	实验设计	13
4.1	与开题评测设计的对应	13
4.2	数据集	13
4.3	对比方法	13
4.4	评测指标	14

4.5	运行环境	14
4.6	实验流程	15
5	实验结果与分析	16
5.1	总体结果	16
5.2	多模态信息的作用	16
5.3	已验证修复点	17
5.4	典型案例分析	17
5.5	剩余错误分析	18
5.6	效率分析	18
5.7	结果小结	18
6	总结、反思与展望	19
6.1	开题目标完成情况	19
6.2	项目调整与反思	19
6.3	系统局限	19
6.4	AGI 视角下的潜力与瓶颈	20
6.5	未来工作	20
6.6	结论	21
	参考文献	22
附录 A	附录：运行与验收说明	23
A.1	后端启动	23
A.2	前端启动	23
A.3	评测命令	23
A.4	提交物	23

1 绪论

1.1 任务背景

随着大语言模型和多模态大模型的发展，面向文档的智能问答系统已经从简单的文本检索逐步走向版面理解、图表推理和可追溯证据定位。大量真实文档并不是线性文本，而是由标题、段落、表格、图像、公式、脚注、页面布局和视觉标记共同组成。对于 PDF、扫描文档、课程讲义、财报、论文和作业说明等材料，关键信息经常以表格行列、图表趋势、局部截图或跨页上下文的形式出现。如果仅将文档抽取为纯文本，再使用传统 RAG 检索，很容易出现三类问题：一是 OCR 或文本抽取丢失视觉布局，二是图表数值和视觉元素无法可靠转写，三是生成答案缺乏可检查的页面证据。

本课程大作业给出的题目 3 要求构建“文档理解 + 多模态检索问答系统”：系统应支持上传多页 PDF 或图像，自动分块文本、表格与图形；支持自然语言问题查询，并在回答中标出引用页面或图号；在 DocVQA、ChartQA 子集或自建数据上评测；并对比纯文本 RAG 与多模态 RAG 的效果差异。期末报告还要求提交最终报告、系统实现细节、代码仓库、定量实验结果、可视化/错误分析以及反思与展望。本文即围绕这些要求，对本项目的系统设计、工程实现、实验评测和局限进行完整总结。

1.2 问题定义

给定一个多页文档 $D = \{p_1, p_2, \dots, p_n\}$ 与用户问题 q ，系统需要从文档中检索出支持回答的证据集合 $E = \{e_1, e_2, \dots, e_k\}$ ，并生成自然语言答案 a 。与普通文本问答不同，本文关注的证据既包括文本片段，也包括页面图像、局部视觉区域、表格行列和图表区域。因此，一个证据单元不仅包含文本内容，还应包含页码、chunk 标识、区域坐标、图像路径和来源类型。系统输出应满足以下目标：

- **正确性**：回答应尽量与标准短答案或人工期望答案一致。
- **可追溯性**：每个答案应标明对应页码、chunk 或视觉区域，便于人工检查。
- **多模态性**：当问题涉及图表、图片、表格或视觉布局时，应使用视觉 evidence，而不是仅依赖 OCR 文本。
- **交互性**：用户可以上传文档、查看解析后的知识库 chunk，并在 PDF 预览中看到对应高亮区域。

1.3 开题目标与完成概览

开题报告中，本项目计划实现三个核心目标：第一，构建多模态 RAG 架构，将文本、表格、图像等信息组织为可检索证据；第二，设计路由决策机制，根据问题类型选择合适的多模态模型或处理路径；第三，提供可解释性溯源，在回答中标注页码、图表或视觉区域。期末阶段，项目围绕这三个目标完成了一个可运行的系统原型，并对部分原计划进行了工程化调整。

总体上，文档解析、多模态 evidence、检索重排、前端上传、PDF 预览、高亮溯源和 DocVQA/ChartQA 评测均已完成；系统还补充了 text、table、figure、code 等 chunk 类型分离、表格 Markdown 入库、统一文本-图像 embedding 接口和 GRPO-style auto router。受课程周期限制，GRPO 路由没有进行真实强化学习训练，而是提供候选模型打分、策略更新钩子和可运行的自动路由入口。这样的调整使系统能够在课程周期内形成稳定闭环，并保留后续继续扩展学习式路由的接口。

1.4 系统贡献

本项目最终实现了一个可运行的多模态文档问答原型，代码仓库包含后端 RAG pipeline、评测脚本、FastAPI 服务层、React 前端和 Gradio 兼容入口。相比开题阶段的系统设想，最终版本从“方案设计”推进到“可评测、可交互、可解释”的工程闭环。本文主要贡献如下：

1. **实现端到端文档 RAG 流程**：系统支持 PDF/图片输入，完成页面渲染、结构化 chunk 构建、向量索引、重排、路由和生成式回答。
2. **引入多模态 evidence**：在文本 evidence 之外，系统加入表格 Markdown、页面级视觉摘要、图像区域和局部高清 crop，并将视觉区域作为可检索证据参与 MM-RAG。
3. **构建真实模型栈**：后端支持 BGE-M3 embedding、FAISS 索引、BGE-reranker-v2-m3 重排和 Qwen3-VL-8B-Instruct 推理，同时保留 mock backend 便于无模型环境下开发调试。
4. **实现可解释前端交互**：React 前端支持上传、解析轮询、知识库 chunk 列表、连续 PDF 预览、chunk 选择与页面高亮联动，接近 RAGFlow 类文档知识库的使用体验。
5. **完成定量评测与错误分析**：在 20 个 DocVQA 与 20 个 ChartQA 样本上比较 Text-RAG 与 MM-RAG，MM-RAG 总体 EM 达到 0.9750，明显优于 Text-RAG 的 0.3750。

1.5 报告结构

本文后续结构如下: 第二章按照 Embedding/Fusion、向量检索、Reranker、GRPO 路由、VLM 与 PDF 高亮等组件说明相关技术与功能对应; 第三章给出系统方法与工程实现; 第四章介绍实验设置、数据集、指标和运行环境; 第五章展示实验结果、案例与错误分析; 第六章总结项目收获, 并从模型局限和 AGI 视角讨论未来方向。

2 相关技术与功能对应

2.1 组件功能概览

开题报告中的系统并不是单一模型，而是由文档解析、Embedding 表征、Fusion 融合、向量检索、Reranker 重排、路由决策、多模态生成和前端溯源预览等组件共同组成。每个组件对应系统中的一个明确功能：文档解析负责把 PDF 转换成可处理的页面、坐标和结构化证据单元；Embedding 负责把文本、OCR 结果和图像特征映射到统一语义空间；Fusion 负责把 text、table、figure、code 等不同类型 chunk 组织到同一个知识库中；向量检索和 Reranker 负责从知识库中找到最相关证据；GRPO 路由对应自动选择问答路径或模型的能力；VLM 负责对图表、图片和表格区域进行视觉理解；前端 PDF 高亮负责让用户检查答案来源。

表 2.1 相关技术组件与项目功能对应

技术组件	对应功能	在本项目中的作用
文档解析与 OCR	将 PDF/图片转成页面、文本、坐标和局部区域。	支撑后续 chunk 构建、PDF 预览和证据高亮。
Embedding	将文本、OCR 结果和图像特征转成向量。	让用户问题和文档 chunk 可以在统一文本-图像空间中匹配。
Fusion	融合 text、table、figure、code、页面图和局部 crop。	将不同来源的证据统一组织为可检索知识单元。
向量检索	从知识库中召回候选证据。	实现长文档问答中的快速 Top-K evidence 搜索。
Reranker	对候选证据重新排序。	过滤弱相关 chunk，提高送入生成模型的证据质量。
GRPO/路由决策	自动判断问题类型并选择合适路径。	实现 GRPO-style auto router、候选 VLM 打分和策略更新钩子。
VLM 多模态生成	理解图表、表格、图片和局部区域。	支撑 MM-RAG 回答视觉问题，弥补纯文本 RAG 不足。
PDF 高亮溯源	展示答案对应页面和区域。	让问答结果可检查，避免系统只输出不可追溯文本。

2.2 Embedding 与 Fusion

Embedding 的作用是把用户问题和文档内容表示成可以比较的语义向量。在普通文本 RAG 中，文档被切成若干文本 chunk，每个 chunk 通过 embedding 模型转换为向量；查询时，用户问题也被转换为向量，再通过相似度搜索找到相关 chunk。这个过程实现了文档问答中的基础召回能力。

但本项目的目标不是只处理纯文本，而是处理 PDF 页面中的表格、图表、图片、代码块和局部视觉区域。因此仅有文本 embedding 不够，还需要 Fusion。这里的 Fusion 指的是把不同来源的信息组织成统一 evidence：页面文本、表格 Markdown、OCR 片段、页面渲染图、局部 crop、页码、坐标和来源类型都会被绑定到同一个证据单元中。这样系统在检索时可以先利用文本语义召回，在生成时再把视觉区域交给 VLM 理解，实现文本检索与多模态证据的融合。

期末实现中，文本 embedding 采用 BGE-M3；图像侧实现了轻量图像特征编码，并保留 Native ViT/CLIP 风格图像编码器的可选接口。最终通过统一文本-图像 embedding 模块将文本向量和图像向量融合到同一检索接口中。该实现比完整训练跨模态大模型更轻量，但已经能体现开题报告中“文本、表格和图像特征统一进入检索问答流程”的核心思想。

2.3 向量检索与 Reranker

向量检索对应的是系统的知识库检索功能。用户上传文档后，系统把文档切分为多个 chunk，并为每个 chunk 建立向量表示；当用户提出问题时，系统通过向量相似度召回 Top-K 候选证据。这一步决定了生成模型能看到哪些材料，是文档问答系统能否答对的前提。

Reranker 的作用是在初步召回之后再次判断候选证据与问题的相关性。向量检索速度快，但可能召回语义接近却并不真正支持答案的片段；Reranker 通过更细粒度的问题-证据匹配，将真正相关的 chunk 排在前面，降低无关材料干扰。对多模态问答而言，Reranker 不只服务于文本答案，也服务于视觉 evidence 的选择：只有先找到正确页面、正确 chunk 或正确区域，VLM 才可能读到正确图表或表格。

因此，向量检索和 Reranker 共同实现了本项目的多模态知识问答基础能力。Text-RAG 依赖它们召回文本证据；MM-RAG 也依赖它们定位页面和候选区域，再进一步使用视觉模型读取页面图像或局部 crop。

2.4 GRPO 与自动路由

开题报告中提出的 GRPO 路由，本质上对应“自动选择问答路径”的功能。不同问题需要不同处理方式：普通定义题适合 Text-RAG；图表数值题适合 ChartQA route；表格行列题需要局部区域 crop；图片、广告和商品标签题需要 VLM 直接读取视觉区域。如果系统能根据用户问题自动选择合适路径，就能避免所有问题都使用同一套低效或不准确的流程。

GRPO 的设想是让轻量路由模型在训练过程中尝试不同选择，再根据回答质

量得到奖励，从而逐步学会为不同问题选择合适的模型或工具。它对应的是更加智能的自动路由能力。受课程周期、训练样本和算力限制，期末实现没有完整训练 Qwen-0.5B 路由器，但实现了 GRPO-style auto router：系统会抽取问题特征，为 Qwen2.5-VL、LLaMA 3.2 Vision 和 DeepSeek-VL 等候选 VLM 计算奖励分数，返回 route、retrieval modes、selected model 和 policy trace，并保留基于 grouped rewards 更新策略表的钩子。

因此，当前 auto 入口已经具备“自动判断问题类型并选择处理路径”的工程表面和可训练接口，只是没有进行真实强化学习训练。报告中将其称为 GRPO 风格自动路由，而不是完整训练后的强化学习模型。

2.5 VLM 多模态生成

VLM 的作用是处理纯文本 RAG 难以解决的视觉理解问题。对于图表、表格、广告、商品标签、截图和复杂页面布局，答案可能并不存在于 OCR 文本中，或者 OCR 文本虽然抽出了一部分内容，却丢失了空间关系。此时需要把页面图或局部 crop 输入视觉语言模型，让模型直接观察图像内容并生成答案。

本项目使用 Qwen3-VL-8B-Instruct 作为主要多模态生成模型^[1]。在实际流程中，系统不会盲目把整页 PDF 全部送入 VLM，而是先用检索和规则定位缩小范围，再把最可能包含答案的局部 crop 作为视觉 evidence。这样可以减少显存和 token 成本，也能让模型注意力集中在图表、表格行或商品 logo 等关键区域。

2.6 PDF 高亮与可解释溯源

开题报告中强调系统需要输出可追溯的页码、图表来源或视觉位置。因此，前端预览不是附加功能，而是文档问答系统可信性的组成部分。用户不仅要看到答案，还要能看到答案来自哪一页、哪一个 chunk、哪一个区域。

本项目参考 RAGFlow 等文档知识库系统的交互方式^[2]：左侧显示知识库 chunk 和问答结果，右侧显示可连续上下翻页的 PDF 预览；当用户选择某个 chunk 或答案引用时，右侧自动定位到对应页并高亮 bbox 区域。这样，chunk 不再是孤立文本，而是和原始 PDF 页面保持联动。该设计让系统更接近真实文档助手，而不是一个只返回文本的问答接口。

2.7 小结

本章按照项目组件说明了相关技术与功能的对应关系: **Embedding** 和 **Fusion** 负责知识表示, 向量检索和 **Reranker** 负责证据召回, **GRPO**/路由负责路径选择, **VLM** 负责视觉理解, **PDF** 高亮负责可解释溯源。第三章将沿着这些组件, 具体说明本项目中各模块如何实现。

3 方法与系统实现

3.1 总体架构

系统总体采用“文档解析-Embedding 与 Fusion-向量检索-Reranker 重排-路由决策-多模态生成-PDF 高亮溯源”的流水线。用户上传 PDF 或图片后，后端首先解析页面文本、页面图、表格、图像和局部区域；随后按 text、table、figure、code 等类型构建结构化 chunk，并通过统一文本-图像 embedding 接口建立索引。查询阶段，系统先通过 auto 路由或手动模式选择 Text-RAG、MM-RAG 或 ChartQA 专用路径，再召回与重排证据，最后由生成模型输出答案和引用信息。前端负责展示知识库 chunk、连续 PDF 预览和答案高亮区域。

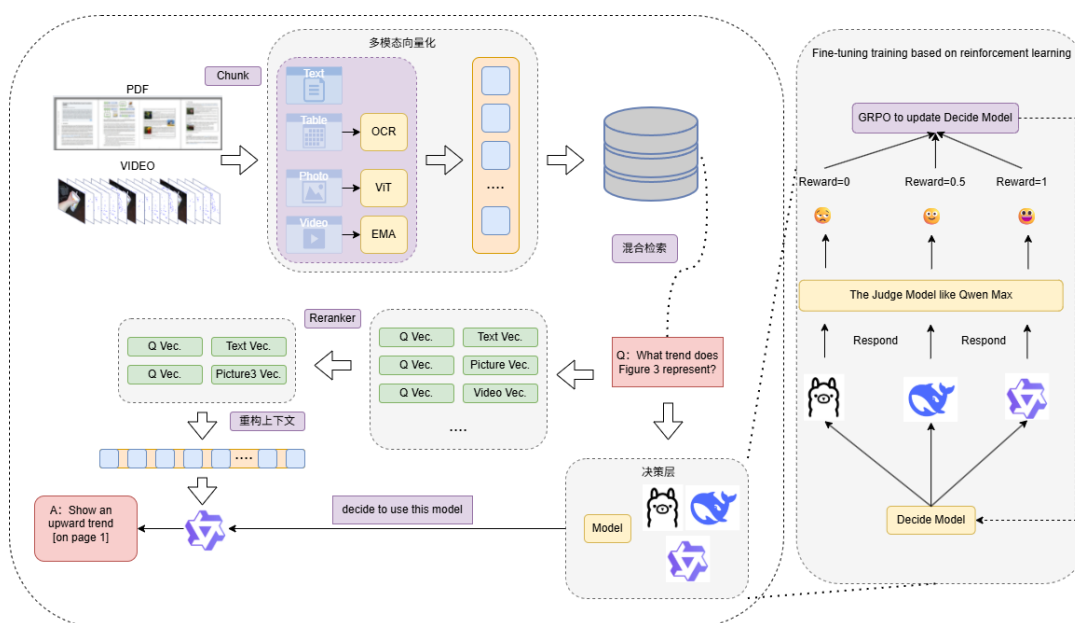


图 3.1 系统总体架构图

图 3.1 展示了开题阶段设计的总体架构。期末实现保留了其中“多模态知识库 + 检索重排 + 路由生成 + 溯源展示”的主线，但做了工程化调整：GRPO 学习式路由没有完整训练，而是实现为 GRPO-style auto router、候选 VLM 打分和策略更新钩子；多模态向量化没有训练大型跨模态模型，而是实现统一文本-图像 embedding 接口、轻量图像特征和可选 Native ViT 图像编码器；前端从单页/单 chunk 展示调整为连续 PDF 预览和区域高亮。

3.2 文档解析实现

文档解析对应第二章中的“文档解析与 OCR”组件，主要由 ingest.py 模块实现。对于 PDF 文件，系统使用 PyMuPDF 抽取每页文本，并将每页渲染为 PNG 页

面图；对于图片文件，系统直接将其视为单页文档。解析后，每一页都包含页码、页面文本、页面图路径、页面宽度和页面高度。

在此基础上，系统将页面内容进一步切分为 `chunk`。文本 `chunk` 只保留普通段落文本；代码段会被识别为 `code chunk`；表格区域会被 PyMuPDF 表格检测抽出并转换为 Markdown table，作为 `table chunk` 入库；页面图片、图表和公式区域会被裁剪为 `figure` 或 `formula chunk`。已识别的表格、图片和公式区域会从普通页面文本中排除，避免同一内容既污染 `text chunk`，又作为结构化 `chunk` 重复出现。这里的 `bbox`、`image path`、`page width` 和 `page height` 都使用普通字段保存，前端可以据此把证据位置映射回 PDF 页面。

文档解析模块解决的是“知识从哪里来”的问题。它不仅为后续 `embedding` 提供文本，也为 VLM 提供页面图和局部视觉区域，是 Text-RAG 与 MM-RAG 共用的基础。

3.3 Embedding 与 Fusion 实现

Embedding 对应语义向量化功能，主要由 `model_stack.py`、`real_models.py` 和 `multimodal_embeddings.py` 管理。真实模型模式下，系统使用 BGE-M3 生成文本 `chunk` 的向量表示，并使用 FAISS 建立向量索引。图像侧默认使用轻量图像特征编码器，将颜色、比例、视觉 `token` 等信息映射为向量；如果配置了图像 `embedding` 模型，也可以使用 Native ViT 图像编码器。用户问题到来后，问题同样被编码为向量，用于和知识库 `chunk` 进行相似度匹配。

Fusion 的实现分为两层。第一层是向量层融合：统一文本-图像 `embedder` 会把文本向量和图像向量按权重融合，并在 `metadata` 中记录 `embedding modalities` 与 `embedding space`。第二层是 `evidence` 层融合：一个 `evidence` 可以同时包含文本内容、页码、页面图路径、`bbox`、OCR 文本、局部 `crop`、表格 Markdown 和来源类型。这样做的好处是：检索阶段可以利用统一向量空间快速召回，生成阶段可以把对应页面图、局部 `crop` 或表格结构交给 VLM 进一步理解。

因此，本项目中的 Fusion 更偏向工程融合：文本负责召回，视觉区域负责补充理解，坐标信息负责溯源展示。它实现了开题报告中“文本、表格和图像信息共同进入多模态 RAG 流程”的目标。

3.4 向量检索与 Reranker 实现

向量检索和 Reranker 对应第二章中的知识问答核心组件，主要在 `pipeline.py` 中组织。系统首先用向量索引召回 Top-K 候选 `evidence`，再使用 BGE-reranker-v2-m3

对问题和候选证据进行二次打分。重排后，系统只保留最相关的证据输入生成模型，减少无关 chunk 对回答的干扰。

在 Text-RAG 路径中，检索和重排直接决定最终上下文；在 MM-RAG 路径中，检索和重排还承担“定位视觉证据”的作用。例如，当问题涉及某个图表、品牌或日程表时，文本检索先帮助系统找到相关页面或相关 OCR 片段，再由视觉区域生成模块裁剪局部 crop。这样，Reranker 不只是提高文本答案质量，也间接提高了 VLM 看到正确区域的概率。

最终 prompt 中，证据会被组织成带编号的片段，如 [E1]、[E2]。模型只需要引用证据编号，后端再把编号映射为具体页码、chunk 和 bbox，避免模型自己编造来源。

3.5 路由决策实现

路由决策对应开题报告中的 GRPO 自动路由模块。原计划中，系统希望通过 GRPO 训练轻量化路由模型，让它根据问题类型选择不同 VLM 或不同处理路径。期末阶段，考虑到训练样本、评分模型和时间限制，项目没有完整训练 Qwen-0.5B 路由器，而是实现了 GRPO-style auto router，作为自动路由的可运行版本。

当前路由逻辑主要分为三类。第一，普通定义、摘要、章节内容查找等问题进入 Text-RAG，直接使用文本 chunk 检索和生成。第二，涉及图片、表格、图表、logo、广告、商品标签或页面区域的问题进入 MM-RAG，系统会优先准备页面图或局部 crop。第三，ChartQA 样本进入图表专用 route，固定生成 chart region，并在 prompt 中加入 compare、difference、max、min、average 等图表任务提示。当前前端默认模型入口为 auto，对应后端自动路由流程；后端会返回 route、retrieval modes、selected model 和 policy trace，便于后续检查和训练。

这一路由实现虽然没有经过真实强化学习训练，但已经完成了“根据问题类型选择合适处理路径”的功能，并保留 grouped rewards 策略更新钩子。后续如果继续扩展 GRPO，只需要把当前 auto 路由产生的路径选择、答案结果和人工/自动评分保存为训练数据，就可以训练更轻量的 route policy。

3.6 VLM 与局部视觉区域实现

VLM 多模态生成主要由真实模型 backend 调用 Qwen3-VL-8B-Instruct 完成。系统不会把所有页面无差别送入 VLM，而是先通过检索、OCR bbox 和问题关键词缩小候选范围，再裁剪局部高清 crop。这样可以降低视觉 token 成本，也能减少页面其他内容对模型的干扰。

局部区域生成主要由 `vision_regions.py` 完成。对于 DocVQA 中的数字、年份、广告品牌、商品标签和日程表问题，系统根据 OCR bbox 和关键词生成候选区域；对于 ChartQA，系统优先裁剪图表主体区域；对于 PDF 中检测到的表格和图片区域，系统会保留裁剪图、结构化 metadata 和可用于前端展示的 preview。实际评测中，局部 crop 对 ChartQA 和视觉实体问题提升明显，说明 VLM 的作用不是简单“多看一张图”，而是需要配合检索和区域选择，把视觉模型的注意力集中在高价值证据上。

3.7 FastAPI 服务层实现

FastAPI 服务层主要由 `api.py` 实现，负责把后端 pipeline 封装为前端可调用的接口。它维护文件状态、解析任务状态、chunk 列表、页面图和问答结果，使前端可以轮询解析进度并展示知识库内容。

表 3.1 FastAPI 主要接口

接口	功能
POST /files	上传一个或多个 PDF/图片文件。
POST /files/id/parse	启动解析、渲染、chunk 构建与索引。
GET /jobs/id	查询解析进度和错误信息。
GET /files/id/chunks	返回知识库 chunk 列表。
GET /pages/image	返回 PDF 页面渲染图。
POST /query	执行检索问答，返回答案、引用和证据。
DELETE /files/id	删除上传文件、处理结果和预览缓存。

服务层支持 `uploaded`、`queued`、`parsing`、`ready` 和 `failed` 等状态。对于真实模型 API 欠费、权限不足或解析失败等情况，后端会返回结构化错误，前端可以显示友好提示，而不是直接暴露原始供应商报错。

3.8 前端 PDF 预览与高亮实现

前端主界面由 `App.tsx` 实现，使用 React、Vite、TailwindCSS 和 Framer Motion。最终界面采用左右双栏：左侧是文件、知识库 chunk 和问答结果，右侧是连续 PDF 预览。用户选中某个 chunk 或答案引用后，右侧 PDF viewer 自动滚动到对应页，并根据 bbox 绘制高亮框。

这个设计对应第二章中的“PDF 高亮与可解释溯源”。与只显示某个 chunk 文本不同，连续 PDF 预览可以让用户上下翻页检查文档上下文；高亮框则把 evidence

从抽象的 `chunk` 还原到原始页面区域。这样用户能判断答案是否真的来自文档，而不是模型凭空生成。

3.9 代码结构

项目代码按前后端和评测脚本组织，主要文件如表 3.2 所示。这里使用普通正文样式列出路径，避免正文中出现突兀的等宽字体。

表 3.2 项目主要代码结构

路径	功能
frontend/	React + Vite 前端。
src/mlmproject/api.py	FastAPI 服务层。
src/mlmproject/ingest.py	文档解析、页面渲染与结构化区域抽取。
src/mlmproject/pipeline.py	RAG 主流程。
src/mlmproject/ model_stack.py	模型栈、统一 embedding 与 backend 切换。
src/mlmproject/ multimodal_embeddings.py	文本-图像统一向量空间。
src/mlmproject/ grpo_router.py	GRPO-style auto router 与策略更新钩子。
src/mlmproject/ table_markdown.py	表格 Markdown 结构化。
src/mlmproject/ real_models.py	BGE 与 Qwen3-VL 适配器。
src/mlmproject/ vision_regions.py	OCR bbox 与局部 crop。
src/mlmproject/evaluation.py	评测流程。
scripts/	批量评测与 demo 脚本。
data/eval/	评测样本与结果。
Report/	开题报告、期末报告和参考文献。

仓库 README 中提供了环境配置、命令行评测、React + FastAPI 联调和 Gradio demo 的运行方式，满足课程对代码仓库和运行说明的要求。

4 实验设计

4.1 与开题评测设计的对应

开题报告中计划使用 DocVQA、ChartQA 子集或自建数据集进行评测，并关注 Recall@K、MRR、ANLS、EM、溯源准确性和响应延迟等指标。期末阶段，项目优先完成 DocVQA 与 ChartQA 两类公开任务的对比实验：DocVQA 用于检验文档页面、表格、广告和票据类信息抽取能力；ChartQA 用于检验图表数值和视觉趋势理解能力。两类数据分别对应开题报告中“文档解析”和“图表/视觉理解”两个重点场景。

由于课程周期有限，最终报告没有扩展到视频片段和大规模自建中文数据集，而是将主要精力放在“纯文本 RAG 与多模态 RAG 的效果差异”这一核心问题上。这样可以直接验证开题方案中多模态 evidence、路由和局部视觉区域是否真正带来收益。

4.2 数据集

为了验证系统在真实文档理解任务中的效果，本文选择 DocVQA 与 ChartQA 两类数据作为评测来源。DocVQA 主要考察模型对文档页面、表格、广告、表单和票据等场景的文字定位与抽取能力；ChartQA 主要考察模型对柱状图、折线图、数值比较和视觉趋势的理解能力。两者分别覆盖文档问答中的文本密集型场景和图表视觉推理场景，符合课程要求中“至少在一个文档数据集上评测，并对比纯文本 RAG 与多模态 RAG”的标准。

最终统计使用 20 个 DocVQA 样本和 20 个 ChartQA 样本，共 40 个样本。由于此前发现 docvqa_49153 存在标注歧义，最终 20+20 统计排除了该样本，并使用新样本替代。评测结果由以下实验目录合并得到：基础 15+15 实验、新增 5+5 实验、三个历史错误修复重测和两个 ChartQA 截断错误修复重测。

4.3 对比方法

本文比较两种主要方法：

- **Text-RAG**：只使用文档文本抽取结果构建 chunk 和向量索引。该方法适合章节总结、定义、普通文本问答等场景，是多模态系统的基础 baseline。
- **MM-RAG**：在 Text-RAG 基础上加入页面视觉 evidence、局部高清 crop、图表专用 route 和 VLM 推理。该方法能处理图表、视觉区域、广告品牌、商品

标签、表格和图像型问题。

两种方法共享同一套短答案抽取、归一化和评分脚本，以保证比较公平。对于生成式回答，系统优先抽取 Final answer: 后的短答案，再进行大小写、标点、数字容差和实体子串归一。

4.4 评测指标

本文使用以下指标：

- **Exact Match (EM)**: 预测短答案与标准答案严格匹配的比例。该指标适合衡量抽取式或短答案问答。
- **ANLS**: 平均归一化 Levenshtein 相似度，是 DocVQA 常用指标，能够容忍轻微拼写或格式差异。
- **Answer Match**: 经过项目内归一化逻辑后的答案匹配率，用于处理实体子串、数值容差和截断修复等情况。
- **Latency**: 单个问题平均推理耗时，用于评估系统可用性。

此外，开发过程中也记录 Recall@K、MRR 和 Citation Accuracy，用于检查检索和 citation 对齐是否稳定。这些指标对应开题报告中的“检索与路由指标”和“溯源准确性”。在最终 20+20 表格中，重点展示 EM、ANLS、Answer Match 和平均延迟；检索指标主要用于开发调试和错误定位，没有作为最终主表展开。

4.5 运行环境

真实模型评测使用如下环境：

- Python: C:/anaconda/envs/first/python.exe
- OCR: EasyOCR CPU 模式，使用 `-no-ocr-gpu`
- Embedding: BGE-M3, CPU 推理
- Reranker: BGE-reranker-v2-m3, CPU 推理
- VLM: Qwen3-VL-8B-Instruct, GPU 推理
- 显存策略: `-max-images 1`, 优先传局部 crop
- 图像限制: 整页最长边 1280, region crop 默认最长边 2048
- 生成长度: `-max-new-tokens 96`

系统开发机主要使用 NVIDIA GeForce RTX 5090D 32GB。考虑到课程项目的演示需求，前端和后端同时保留 mock 模式，使无 GPU 环境下仍可展示上传、解析、预览和 API 流程。

4.6 实验流程

实验流程分为四步。第一步，使用统一脚本读取 DocVQA 和 ChartQA 样本，并构建文档对象。第二步，分别运行 Text-RAG 和 MM-RAG，保存每个样本的预测答案、top evidence、引用页、route、延迟和失败标签。第三步，对模型输出进行短答案抽取与归一化，计算 EM、ANLS 和 Answer Match。第四步，对错误样本进行人工检查，分析错误来源是否来自检索、视觉区域选择、OCR、VLM 推理或答案截断。

为减少偶然性，项目在最终统计前对若干错误进行了定点修复与重测。例如，针对 ChartQA 截断年份和 Yes/No 比较题，系统加入了短答案修复逻辑；针对广告品牌和商品标签题，系统加入 logo 区域 crop；针对图表问题，系统固定生成 chart region 并增强图表任务提示。

5 实验结果与分析

5.1 总体结果

表 5.1 展示了 20 个 DocVQA 与 20 个 ChartQA 样本上的最终评测结果。可以看到，MM-RAG 在两个数据集上都显著优于 Text-RAG。总体上，MM-RAG 的 EM 和 ANLS 均达到 0.9750，而 Text-RAG 的 EM 为 0.3750，ANLS 为 0.4265。

表 5.1 Text-RAG 与 MM-RAG 在 DocVQA/ChartQA 子集上的结果

Dataset	Mode	Count	EM	ANLS	Answer Match	Avg Latency ms
DocVQA	MM-RAG	20	0.9500	0.9500	0.9500	3858.81
DocVQA	Text-RAG	20	0.5500	0.6230	0.6500	5697.80
ChartQA	MM-RAG	20	1.0000	1.0000	1.0000	3352.50
ChartQA	Text-RAG	20	0.2000	0.2300	0.2500	3451.81
Overall	MM-RAG	40	0.9750	0.9750	0.9750	—
Overall	Text-RAG	40	0.3750	0.4265	0.4500	—

在 DocVQA 中，Text-RAG 已能处理一部分 OCR 文本清晰、答案直接出现在页面文字中的问题，因此 EM 达到 0.5500。但对于需要结合页面布局、广告品牌、商品标签或表格行列定位的问题，Text-RAG 容易召回不完整文本或错误上下文。MM-RAG 通过局部视觉区域和 VLM 推理，将 DocVQA EM 提升到 0.9500。

在 ChartQA 中，Text-RAG 表现更弱，EM 仅为 0.2000。这符合预期，因为图表中的数值、趋势和比较关系往往无法被 OCR 完整表达。MM-RAG 固定使用 chart region，并在 prompt 中加入图表任务提示，最终在当前 20 个 ChartQA 样本上达到 20/20。

5.2 多模态信息的作用

从结果看，多模态信息主要在以下三类问题中发挥作用：

- 图表数值与趋势问题：**ChartQA 中的最大值、最小值、差值、平均值和 Yes/No 比较题，都需要读取图表视觉结构。局部 chart crop 使 VLM 能直接观察坐标轴、柱高或折线趋势。
- 广告和商品标签问题：**DocVQA 中的商品名、品牌 logo 和广告主视觉信息有时不会被 OCR 完整抽出。局部 logo crop 可以补充 OCR 缺失。
- 页面区域定位问题：**对于表格、时间表、票据和版面型问题，视觉区域提供了文字之间的空间关系，有助于判断答案所在行列。

这说明多模态 RAG 的优势并不只是“多传一张图片”，而是通过区域选择把视觉模型的注意力集中到高价值证据上。相比整页输入，局部 crop 在显存、延迟和答案准确性之间取得了更好的平衡。

5.3 已验证修复点

最终 20+20 结果中纳入了多项修复：

- **短答案抽取与评分归一**：优先使用 Final answer:；支持实体子串匹配；支持 ChartQA 数值容差；支持截断年份和截断 Yes/No 比较题修复。
- **局部高清 crop**：OCR bbox 写入结果详情；DocVQA 支持数字、年份、广告品牌和时间表 row crop；MM-RAG top-k 排序参考区域 metadata score。
- **ChartQA 专用 route**：固定生成 chart region，并根据问题类型加入 count、difference、compare、min/max、average 等提示。
- **证据引用对齐**：模型只输出 [E1] 等编号，后端统一生成 page/chunk citation，避免模型自写来源导致结构化引用不一致。

5.4 典型案例分析

表 5.2 给出若干关键修复样例。它们说明系统性能提升来自多个环节共同作用，而不是单一模型替换。

表 5.2 典型修复案例

Sample	Gold	修复后输出	修复效果
docvqa_57364	dark fantasy	Dark Fantasy Choco Fills	实体粒度归一后正确。
docvqa_57366	wills lifestyle	WILLS LIFESTYLE	广告/商品 logo crop 后正确。
chartqa_000013	21.6	21.7	relaxed numeric tolerance 后归一为正确。
chartqa_000017	2011	Final answer: 201	截断年份修复后恢复为 2011。
chartqa_000019	No	计算过程截断	Yes/No 比较兜底后恢复为 No。

这些案例体现了最终系统的一个重要特点：文档问答不能只依赖生成模型本身，还需要围绕文档结构设计证据选择、答案抽取和评分归一。尤其在课程项目这样的小样本评测中，错误往往不是模型“完全不懂”，而是区域选择、输出截断或答案格式导致的。

5.5 剩余错误分析

最终 MM-RAG 仍有一个主要错误样本：docvqa_49181。问题为 “What session is at 2.00 to 5.00p.m.?”，标准答案为 “trrf scientific advisory council meeting”，而 MM-RAG 输出为另一个不完整 session。人工检查发现，Text-RAG 在该题中可以答对，说明 OCR 文本里存在正确答案；MM-RAG 出错的原因是局部视觉 crop 选到了错误或不完整的表格上下文。

该错误暴露出当前系统在复杂表格和日程表上的局限。对于“时间范围到会议名称”的问题，系统需要先定位时间单元格，再横向扩展到同一行的 session 名称。如果只依据关键词或局部 OCR bbox 裁剪，容易截断左侧时间列或右侧名称列，导致 VLM 读错行。因此后续应增加结构化表格行重建：将同一 y-band 的 OCR box 合并成候选行，并在检索阶段优先召回包含时间 anchor 和完整 session 文本的行。

5.6 效率分析

从延迟看，MM-RAG 在当前 20+20 评测中并没有显著慢于 Text-RAG。原因在于最终策略优先传入单个局部 crop，而不是整页多图输入。早期完整页面视觉摘要构建一篇 11 页 PDF 约需 146.6 秒，首个页面冷启动约 24 秒，后续每页约 8-9 秒；而局部 crop 推理更适合在线问答场景。未来如果要处理更长文档，可以将页面视觉摘要作为离线索引构建步骤，将在线查询阶段控制在少量高价值 region 上。

5.7 结果小结

总体而言，实验验证了本文系统设计的有效性。Text-RAG 能作为稳定 baseline，但在图表、视觉区域和复杂版面中能力有限；MM-RAG 通过局部视觉 evidence、图表路由和 VLM 生成显著提升了回答准确性。当前系统已经达到课程最低验收标准，并在可解释证据展示、前后端联调和错误分析方面形成了完整闭环。

6 总结、反思与展望

6.1 开题目标完成情况

本文围绕开题报告中提出的多模态文档智能助手目标，完成了一个从后端模型 pipeline 到前端交互界面的系统原型。系统能够上传 PDF 或图片，解析页面，构建文本和视觉 evidence，使用 Text-RAG 或 MM-RAG 进行问答，并在前端展示连续 PDF 预览与 evidence 高亮。实验表明，多模态 evidence 对文档问答具有明显价值：在 40 个 DocVQA/ChartQA 样本上，MM-RAG 总体 EM 达到 0.9750，显著超过 Text-RAG 的 0.3750。

对照开题报告，最终完成情况可以概括为四点。第一，**多模态 RAG 架构**已经落地：系统完成 PDF/图片解析、text/table/figure/code 等结构化 chunk 构建、页面图、局部 crop、向量检索、重排和生成式回答。第二，**视觉定位与溯源**已经落地：每个 evidence 保留页码、chunk 和 bbox，前端可以在 PDF 预览中高亮对应区域。第三，**评测与报告**已经完成：项目对比了 Text-RAG 与 MM-RAG，并给出了定量结果、案例和错误分析。第四，**路由决策**完成了工程化落地：原计划中的强化学习路由没有进行真实训练，但系统实现了 GRPO-style auto router、候选 VLM 打分、retrieval modes 选择和策略更新钩子。

6.2 项目调整与反思

相比开题阶段的设想，最终项目主要有三处调整。第一，视频片段理解没有作为期末主要评测对象，项目将重点收敛到 PDF 与图片文档，这与课程题目中的文档问答场景更直接对应。第二，多模态向量化没有训练大型跨模态检索模型，而是实现统一文本-图像 embedding 接口、轻量图像特征和可选 Native ViT 图像编码器，使系统更容易在单机环境下稳定运行。第三，多模型自动选择没有训练真实 Qwen-0.5B 路由器，而是通过 GRPO-style auto router 暴露可训练接口，并在演示阶段用确定性特征、候选奖励和策略钩子完成自动路径选择。虽然这些调整降低了训练复杂度，但提高了项目完成度、可复现性和演示稳定性。

6.3 系统局限

当前系统仍存在以下局限：

- **复杂表格结构理解仍有限**：系统已经支持表格区域抽取和 Markdown 入库，但对跨列、跨行、合并单元格和复杂日程表的行列语义恢复仍不稳定。

- **OCR 噪声影响检索**：当 OCR 漏字、错字或顺序混乱时，文本 chunk 和区域 anchor 都会受影响。视觉模型可以弥补部分问题，但如果 crop 本身错误，仍会生成错误答案。
- **视觉区域检测仍偏规则化**：当前 region 生成主要针对实验样本中常见问题类型，泛化到任意文档时可能需要更强的 layout detector、表格检测器或端到端视觉检索模型。
- **模型调用成本和延迟**：Qwen3-VL 对整页视觉摘要的处理仍较慢。在线问答必须控制输入图片数量和分辨率，否则会显著增加延迟与显存压力。
- **评测规模有限**：当前 20+20 样本可以验证主要功能和改进方向，但不足以代表全部文档类型。未来需要更大规模、更细粒度的评测集合。

6.4 AGI 视角下的潜力与瓶颈

从 AGI 视角看，文档智能助手是多模态大模型落地的重要场景。真实办公和科研环境中的知识并不只存在于纯文本中，而是混合在 PDF、图表、表格、扫描件、网页截图和演示文稿中。一个更接近通用智能的系统，需要像人一样阅读页面：先观察版面结构，再定位证据区域，理解文本与图表之间的关系，最后给出带来源的结论。本文系统在小规模上验证了这一方向的可行性。

但当前系统距离真正的 AGI 文档助手仍有明显距离。首先，模型仍然依赖外部规则帮助其选择区域，还不能自主形成稳定的阅读策略。其次，模型对复杂表格和跨页推理的鲁棒性不足，缺乏长期记忆和全局任务规划。再次，生成模型虽然能给出流畅解释，但仍可能在证据不足时过度推断。因此，未来的文档智能系统不仅需要更强 VLM，也需要更可靠的工具调用、结构化解析、检索验证和自我纠错机制。

6.5 未来工作

后续可以从以下方向改进：

1. **结构化表格解析**：引入表格检测和行列重建，将 OCR box 合并为结构化 Markdown 或 DataFrame，提升日程表、财报表格和复杂表格问答能力。
2. **视觉检索模型**：尝试 ColPali 类原生视觉检索方法，减少对 OCR 文本的依赖，并提升复杂版面召回质量。
3. **学习式路由**：在规则路由稳定后，收集更多查询和答案反馈，用轻量模型学习 query 到 route、crop 类型和 evidence 类型的映射。
4. **更大规模评测**：扩展到更多 DocVQA、ChartQA、TextVQA 和自建中文文档

样本，报告不同文档类型下的细分性能。

5. **前端产品化**：继续优化 PDF viewer、chunk 管理、多文件会话、证据编辑和失败重试，使其更接近真实知识库产品。

6.6 结论

本文完成了一个多模态文档理解问答系统，并通过定量实验验证了多模态 RAG 相比纯文本 RAG 的优势。系统在工程上实现了文档上传、解析、索引、检索、生成、引用和 PDF 高亮预览，在实验上达到了较高的 DocVQA/ChartQA 子集准确率，在分析上明确了复杂表格定位和长文档视觉成本等后续挑战。整体来看，本项目满足课程期末验收对系统实现、定量实验、错误分析和反思展望的要求，也为后续构建更通用、更可靠的文档智能助手提供了基础。

参 考 文 献

- [1] Qwen Team, “Qwen3-vl: Multimodal large language models for visual understanding,” 2025.
[Online]. Available: <https://github.com/QwenLM/Qwen3-VL>
- [2] Infinity Team, “Ragflow: Deep document understanding based open-source rag engine,” 2024.
[Online]. Available: <https://github.com/infiniflow/ragflow>

附录 A 附录：运行与验收说明

A.1 后端启动

真实模型联调时，可使用如下命令启动后端：

```
$env:MLLMPROJECT_USE_REAL_MODELS="1"  
$env:MLLMPROJECT_QWEN3_MODEL_PATH="model"  
python -m uvicorn mllmproject.api:app --host 127.0.0.1 --port  
8000
```

无 GPU 或不需要真实模型时，可使用默认 mock backend 运行前后端流程。

A.2 前端启动

```
cd frontend  
npm.cmd install  
npm.cmd run dev
```

浏览器访问 <http://127.0.0.1:5173>，即可使用上传、解析、知识库 chunk、PDF 预览和问答功能。

A.3 评测命令

```
python scripts/run_benchmark_eval.py --mode text-rag  
python scripts/run_benchmark_eval.py --mode mm-rag
```

评测结果保存到 `data/eval/benchmarks/` 与 `data/eval/results/`。最终报告中的 20+20 结果来自仓库内已保存的评测输出。

A.4 提交物

本项目提交物包括：

- 项目代码仓库与 README 运行说明。
- React + FastAPI Web Demo。
- 命令行评测脚本与实验结果。
- 开题报告、期末报告与展示 PPT。
- Demo 录屏材料。